BrainAI

ProSuite Industrial

PROSUITE INDUSTRIAL

PRODUKT-DOKUMENTATION

BrainAI (Inhaber: Sascha Alexander Köhne)

BrainAI Inhaber: Sascha Alexander Köhne	PROSUITE INDUSTRIAL	🟢 ÖFFENTLICHE KLASSIFIZIERUNG
Core Architecture & Development	**Technical Documentation**	Zugriffsstufe: Öffentlich / Beta

---

Revisionshistorie

Version	Datum	Autor	Status	Beschreibung
1.0.0	28.02.2026	S. Köhne	Stable	Initial Release: Modular JS Architecture (Zero-Trust)

1. Einleitung: Die ProSuite JS-Architektur

Das JavaScript-Ökosystem der ProSuite (ProWB Edition) wurde nach dem strikten **Separation of Concerns (SoC)** Prinzip und unter **Zero-Trust-Prämissen** von Grund auf neu strukturiert.

Im Gegensatz zu klassischen Web-Anwendungen (React, Angular), bei denen UI, Routing und Logik stark verwoben sind, ist die ProSuite in **7 hochspezialisierte, voneinander isolierte Module** unterteilt. Diese Architektur garantiert, dass Speicherlecks, XSS-Angriffe oder DOM-Abstürze ausgeschlossen werden und die Kryptografie absolut unangreifbar bleibt.

Die Dateien werden vom ProWB (C-Builder) in aufsteigender numerischer Reihenfolge (`00_` bis `60_`) in den `index.html` Monolithen injiziert, um eine fehlerfreie Initialisierung zu gewährleisten.

---

2. Modul-Spezifikationen ("Wo ist was?")

`00_bridge.js` (Host-Schnittstelle)

Zuständigkeit: Bidirektionale Kommunikation mit potenziellen Host-Systemen.

- Agiert als entkoppelte Schicht zwischen dem Web-Frontend und nativen Containern (WASM-Host, MAUI, WebView2).
- Nutzt CustomEvents (`bridge-data`), um eingehende Daten im Browser zu verteilen, ohne harte Abhängigkeiten zu anderen Skripten zu erzwingen.

`10_base_app.js` (Boot & Core Loader)

Zuständigkeit: Globaler State und Initialisierung.

- Definiert den globalen Namensraum `app` und `app.state` (wo z.B. der `activeKey` temporär im RAM gehalten wird).
- Liest die C-Variable `PRO_WASM_CORE` aus (welche vom ProWB injiziert wird) und lädt dynamisch den passenden WebAssembly-Kern (`prokey.js`, `proed.js` etc.).

- Feuert das kritische Event `app-ready`, sobald der Boot-Prozess abgeschlossen ist.

`20_navigation.js` (Routing & History)

Zuständigkeit: URL-Handling und View-Switching.

- Kapselt die `app.navigate(viewId)` Funktion.
- Schaltet die Sichtbarkeit der vom ProWB generierten `<section>` Blöcke (Views und Markdown-Docs) um.
- **Sicherheits/UX-Feature:** Beinhaltet den Scroll-Fix, der sicherstellt, dass beim Aufruf eines Handbuchs (`doc_...`) automatisch nach ganz oben gescrollt wird.
- Verwaltet die Browser-Historie (`pushState`), damit die "Zurück"-Taste des Browsers reibungslos funktioniert.

`30_ui_general.js` (Theme & UI Management)

Zuständigkeit: Rein visuelle Logik ohne kryptografische Relevanz.

- Beinhaltet die Logik zum Umschalten der Themes (`light`, `matrix`, `proedc`) via `document.body.setAttribute`.
- Steuert das Tab-Switching (z. B. Wechsel zwischen Text- und Datei-Eingabe).
- Handhabt die Auto-Swap-Logik für Ein- und Ausgabefelder, wenn der Benutzer zwischen "Encrypt" und "Decrypt" wechselt.

`40_crypto_core.js` (WASM Bindings & Kryptografie)

Zuständigkeit: Die Brücke zu den C-Funktionen.

- **Heap-Management:** Enthält `getMem()`, um direkten Zugriff auf den `HEAPU8` (WASM-Arbeitsspeicher) zu erhalten.
- **Konvertierung:** Hex-to-Byte und Byte-to-Hex Logik (`toHex`, `fromHex`).
- **WASM-Calls:** Hier liegen die Funktionen `handleNewSession` (ProKey), `computeProHash` (ProHash) und `executeProED` (Text & File Encryption).
- **Zero-Trust Enforcement:** Jeder Aufruf ist in `try/catch/finally`-Blöcke gehüllt. Im `finally`-Block wird zwingend `ProSecurity.secureFree()`

aufgerufen, um sicherzustellen, dass Pointer und Daten im RAM vernichtet werden, selbst wenn die C-Engine einen Fehler wirft.

`50_terminal.js` (Logging & Ring-Buffer)

Zuständigkeit: Visuelles Audit-Feedback für den Endanwender.

- Kapselt die Konsolen-Ausgaben in das HTML-Terminal der ProSuite.
- **XSS-Schutz:** Jeder Log-Eintrag wird durch die interne Funktion `_escapeHtml` maskiert.
- **DOM-Protection (Ring-Buffer):** Limitiert die Anzeige auf maximal 100 Zeilen (`MAX_LINES`). Dadurch wird verhindert, dass bei extrem schnellen Audit-Läufen (z.B. durch ProTU) der Browser aufgrund zu vieler HTML-Elemente einfriert.

`60_security.js` (Zero-Trust Panic & Memory Wipe)

Zuständigkeit: Physische Speichersicherheit und Hard-Reset.

- **`secureFree(ptr, size)`:** Der wichtigste Baustein der Suite. Überschreibt den allokierten Speicherbereich im WASM-Heap physisch mit Nullen (`mem.fill(0)`), bevor er in Emscripten freigegeben wird.
- **`panicWipe()`:** Die "Atomoption". Überschreibt den Session-Key im RAM mit Nullen, löscht den `localStorage`, resettet alle Formularfelder im DOM und erzwingt einen Hard-Reload der Seite.
- **`checkAutoLoad()`:** Verifiziert gespeicherte Schlüssel aus dem LocalStorage auf exakte Byte-Längen (32, 64, 128) und mountet sie sicher in den State.

3. Event-Driven Architecture (Entkopplung)

Um Abhängigkeiten (Spaghetti-Code) zu vermeiden, kommunizieren die Module primär über native DOM-Events. Ein Modul muss nicht wissen, ob ein anderes Modul existiert.

Event-Name	Auslöser	Reaktion / Listener
`app-ready`	`10_base_app.js` (Nach Boot)	Alle anderen Skripte registrieren hierüber ihre UI-Listener (`onclick`).
`core-ready`	`10_base_app.js` (Nach WASM load)	`50_terminal.js` loggt den erfolgreichen Start der C-Engine.
`security-wipe-complete`	`60_security.js` (Nach Panic)	Trigger für UI-Updates vor dem finalen Reboot.

4. Code-Hygiene & Speicher-Vektoren

- **Globale Variablen:** Der einzige globale Namensraum ist das Objekt `app` sowie die klar definierten Konstanten `ProUI`, `ProCrypto`, `ProTerminal` und `ProSecurity`.
- **Garbage Collection Vermeidung:** Sensible Daten (wie Schlüssel oder Klartexte) werden **nicht** dem JavaScript Garbage Collector überlassen, da JS keine Garantien zur Löschdauer gibt. Stattdessen werden sie manuell in den C-Heap (`_malloc`) geschoben, dort verarbeitet und per `secureFree` aktiv mit Nullen überschrieben.

© 2026 Sascha Köhne	✉️ **Contact for KYC:** koehne83@googlemail.com	🛡️ **ProTU-Verified**

System Architect	BrainAI Inhaber : Sascha Alexander Köhne	We don't need **BRUTEFORCE**, we know **PHYSICS**